

Univerzitet u Beogradu
Fakultet organizacionih nauka

Projektni rad iz predmeta

**Alati i metode veštačke inteligencije i softverskog
inženjerstva**

Mentor:

dr Dragan Đurić

Student:

Natalija Branković 2024/3814

Beograd, 2026

Sadržaj

1. Korišćenje AI alata	3
1.1. Pomoć pri razumevanju biblioteka i implementaciji neuronske mreže (preprocess.clj, model.clj, train.clj, persistence.clj, forecast.clj)	3
1.2. Generisanje opisnog sadržaja (allergen_info.clj)	3
1.3. Pomoć pri pisanju testova (core_test.clj)	3
1.4. Pomoć pri pisanju dokumentacije	4
2. Opis aplikacije	5
3. Slučajevi korišćenja	6
4. Prikaz aplikacije	8
4.1 Početni ekran i autentifikacija	8
4.2 Registracija	8
4.3 Glavni meni	10
4.4 Trenutni nivo polena	10
4.5 7-dnevna prognoza polena	11
4.6 Da li da izađem danas	11
4.7 Top 5 gradova sa najnižim polenom	12
4.8 Lični kalendar alergija	13
4.9 Informacije o alergenu	13
4.10 Izmena profila	14
5. Implementacija	15
6. Testiranje	27

1. Korišćenje AI alata

Tokom razvoja PollenCast projekta, AI alati su korišćeni isključivo kao podrška za brže savladavanje sintakse, razumevanje kompleksnijih koncepata i generisanje repetitivnog sadržaja. Sve ključne arhitekturne odluke, dizajn algoritama, logika računanja rizika, struktura neuronske mreže i celokupan sistem osmišljeni su i implementirani samostalno.

1.1. Pomoć pri razumevanju biblioteka i implementaciji neuronske mreže (*preprocess.clj*, *model.clj*, *train.clj*, *persistence.clj*, *forecast.clj*)

Fokus u korišćenju veštačke inteligencije u okviru ovog dela bio je na savladavanju specifične sintakse Deep Diamond i Neanderthal biblioteka za rad sa tenzorima, kao i na povezivanju sa Java NIO paketima za čuvanje modela na disku. AI alati su poslužili za lakše usvajanje načina transformacije podataka i njihovog mapiranja u native tenzore.

Sve ključne odluke donete su potpuno samostalno kroz eksperimentalne iteracije sa ciljem da se greška predikcije svede na minimum. To uključuje definisanje celokupne arhitekture neuronske mreže, konfigurisanje procesa treniranja, obradu datuma kroz ciklično kodiranje, dizajn strukture kliznih prozora za podatke, kao i logiku za memorijsko keširanje modela i prevazilaženje kašnjenja API-ja.

1.2. Generisanje opisnog sadržaja (*allergen_info.clj*)

AI je korišćen za generisanje opisa biljaka, tipičnih alergijskih simptoma i praktičnih saveta za smanjivanje izloženosti za svaku od 26 vrsta alergena polena. Struktura ovih podataka, način integracije sa ostatkom aplikacije, kao i odluka o oslanjanju na specifične izvore poput Wikipedije i European Aeroallergen Network platforme, realizovani su samostalno.

1.3. Pomoć pri pisanju testova (*core_test.clj*)

Prilikom pisanja testova, AI je korišćen isključivo za generisanje inicijalne strukture i osnovnih šablona. Svi testovi pregledani su i adaptirani ručno.

1.4. Pomoć pri pisanju dokumentacije

Tekst ove projektne dokumentacije napisan je samostalno, dok je veštačka inteligencija primenjena tokom procesa pisanja isključivo kao pomoć za formatiranje strukture i ispravljanje pravopisnih i gramatičkih grešaka.

2. Opis aplikacije

PollenCast je konzolna aplikacija razvijena u programskom jeziku Clojure, namenjena praćenju i predviđanju nivoa polena u Srbiji. Aplikacija koristi podatke sa API-ja Agencije za zaštitu životne sredine Srbije koji vraća dnevne nivoe polena za 26 alergena na više lokacija u Srbiji.

Podaci o koncentracijama polena preuzeti su sa platforme dat.gov.rs, konkretno iz skupa podataka "Polen - objedinjeni podaci od 2016. godine". Ovaj skup podataka sadrži resurse o koncentracijama polenovih zrna po danima i lokacijama, kao i tipove alergena sa detaljima o svakoj biljnoj vrsti. Podaci su dostupni putem REST API-ja čija je Swagger specifikacija objavljena na adresi <http://77.46.150.200/api/opendata/schema>. Aplikacija koristi endpoint `GET /api/opendata/pollens/{year}/` koji vraća sve koncentracije polena za određenu godinu, i `GET /api/opendata/locations/` koji vraća listu svih mernih stanica. Svaki zapis sadrži datum merenja, naziv lokacije i vrednosti za 26 biljnih vrsta izražene u pz/m3 (broj polenovih zrna po kubnom metru vazduha).

Pored podataka o koncentracijama, korišćen je i resurs "Tipovi alergena" koji za svaku biljnu vrstu sadrži narodne nazive na srpskom i engleskom jeziku, tip alergena, donju i gornju vrednost koncentracije, period emitovanja polena i stepen alergenosti. Na osnovu ovog resursa implementiran je sistem rangiranja alergenosti biljaka u aplikaciji, koji se koristi pri računanju personalnog rizika za korisnika. Za potrebe treniranja neuronske mreže preuzeti su podaci od 2019. do 2025. godine za 12 odabranih gradova u Srbiji.

Korisnik se pri prvom pokretanju registruje i pravi svoj nalog, gde bira grad u kome živi i alergene na koje reaguje. Na osnovu tog profila, aplikacija nudi sledeće funkcionalnosti: prikaz današnjeg nivoa polena, 7-dnevnu prognozu baziranu na neuronskoj mreži, personalizovanu preporuku da li je bezbedno izaći napolje, prikaz top 5 gradova sa najnižim polenom, lični alergijski kalendar, informacije o svakom alergenu, izmenu profila i odjavu.

Za potrebe predviđanja vrednosti polena, istrenirana je višeslojna neuronska mreža (MLP - Multilayer Perceptron) sa tri skrivena sloja od 512, 256 i 128 neurona. Mreža prima kao ulaz vrednosti koncentracija polena i vremenske karakteristike (mesec, dan u godini i dan u nedelji, predstavljene kroz sinusnu i kosinusnu funkciju) za poslednjih 30 dana, a predviđa koncentracije

za narednih 21 dan. Trenirana je na podacima od 2019. do 2025. godine i postiže srednju kvadratnu grešku korena (Root Mean Square Error - RMSE) od približno 2 pz/m³.

Za razvoj aplikacije korišćene su sledeće biblioteke:

- **Deep Diamond** - biblioteka za duboko učenje, korišćena za definisanje i treniranje neuronske mreže
- **Neanderthal** - biblioteka za linearnu algebru, korišćena ispod Deep Diamond-a
- **Intel MKL** - biblioteka za matematičke operacije
- **Codax** - lokalna key-value baza podataka za čuvanje korisničkih podataka
- **clj-http** - HTTP klijent za komunikaciju sa API-jem
- **Cheshire** - biblioteka za parsiranje JSON odgovora

3. Slučajevi korišćenja

1. Registracija korisnika - Novi korisnik pokreće aplikaciju i bira opciju za registraciju. Unosi korisničko ime, bira jednu od 12 ponuđenih lokacija i selektuje alergene na koje reaguje iz liste od 26 vrsta. Nakon uspešne registracije, korisnik je automatski prijavljen i prelazi na glavni meni.

2. Prijava korisnika - Registrovani korisnik unosi korisničko ime i prijavljuje se u aplikaciju. Aplikacija učitava korisnikov profil iz lokalne baze i prikazuje glavni meni sa gradom i korisničkim imenom u zaglavlju.

3. Odjava korisnika - Prijavljeni korisnik bira opciju Logout iz glavnog menija. Aplikacija ga odjavljuje i vraća na početni meni za prijavu i registraciju, bez zatvaranja aplikacije.

4. Prikaz trenutnog nivoa polena - Korisnik bira opciju za prikaz trenutnog nivoa polena. Pošto API koristi manuelno unete podatke koji kasne nekoliko dana, aplikacija prikazuje datum poslednjeg dostupnog merenja, a zatim koristi istreniranu neuronsku mrežu da predvidi vrednosti polena za današnji dan. Prikazuju se sve vrste polena sa vrednošću iznad 5 pz/m³, sortirane od najveće ka najmanjoj, sa nivoima NONE, LOW, MEDIUM, HIGH i VERY HIGH prema

pragovima koncentracije. Na kraju se posebno prikazuju korisnikovi alergeni sa njihovim vrednostima.

5. 7-dnevna prognoza polena - Korisnik bira opciju za prognozu polena. Aplikacija učitava poslednjih 30 dana merenja sa API-ja i prosleđuje ih neuronskoj mreži koja generiše predikcije za narednih 21 dan od dana poslednjeg dostupnog merenja. Prikazuje se 7 dana počevši od današnjeg datuma. Za svaki dan prikazuju se vrste polena sa vrednošću iznad 5 pz/m³, sortirane po vrednosti, uz poseban prikaz vrednosti korisnikovih alergena.

6. Da li da izađem danas - Korisnik bira opciju za personalizovanu preporuku. Aplikacija koristi predikciju neuronske mreže za današnji dan i za svaki korisnikov alergen računa individualni rizik kombinovanjem koncentracije polena i stepena alergенosti te biljke. Ukupni rizik se određuje prema najgorem pojedinačnom alergenu i prikazuje se kao NONE, LOW, MEDIUM, HIGH ili VERY HIGH, uz konkretan savet.

7. Top 5 gradova sa najnižim polenom - Korisnik bira opciju za prikaz gradova sa najnižim nivoima polena. Aplikacija koristi neuronsku mrežu da predvidi vrednosti polena za današnji dan za sve podržane gradove, sabira vrednosti svih 26 vrsta polena i sortira ih od najmanje ka najvećoj ukupnoj vrednosti. Prikazuje se rang lista top 5 gradova sa najmanjim ukupnim nivoom polena, kao i vrednosti korisnikovih alergena za svaki grad.

8. Lični kalendar alegija - Korisnik bira opciju za kalendar alergija. Aplikacija prikazuje koji od korisnikovih alergena su aktivni u trenutnom mesecu, sa sezonom cvetanja i stepenom alergенosti. Zatim prikazuje godišnji pregled po mesecima, gde je vidljivo kada je koji alergen aktivan tokom godine. Tekući mesec je posebno označen.

9. Informacije o alergenu - Korisnik bira opciju za informacije o alergenima i iz liste od 26 vrsta bira onu o kojoj želi da sazna više. Aplikacija prikazuje detaljan opis biljke, tipične alergijske simptome koje izaziva i praktične savete za smanjivanje izloženosti polenu te biljke.

10. Izmena profila - Prijavljeni korisnik bira opciju za izmenu profila. Može da promeni grad ili da ažurira listu alergena. Nakon izmene, promene se odmah primenjuju i korisnik nastavlja rad sa ažuriranim profilom.

4. Prikaz aplikacije

4.1 Početni ekran i autentifikacija

Pri pokretanju aplikacije komandom **lein run**, prikazuje se zaglavlje sa nazivom aplikacije i meni u kojem se korisniku nude mogućnosti za prijavu, registraciju i izlazak iz aplikacije.

```
=====
POLLENCAST
Pollen Forecast Monitor for Serbia
=====

1. Login
2. Register
0. Exit
```

4.2 Registracija

Odabirom opcije za registraciju unosom broja 2 u konzolu, korisnik unosi željeno korisničko ime i bira grad u kojem se nalazi. Aplikacija trenutno podržava 12 mesta u Srbiji, pri čemu postoje dve lokacije u Beogradu.

```
--- REGISTER ---
Choose a username: natalija_br

Select your city:
1. Beograd - Zeleno Brdo
2. Beograd - Novi Beograd
3. Kikinda
4. Vrbas
5. Vrsac
6. Zajecar
7. Kragujevac
8. Krusevac
9. Nis
10. Vranje
11. Kraljevo
12. Cacak

> 2
```


Zatim korisnik bira alergene na koje reaguje, radi personalizovanog iskustva.

```
Select your allergens:  
(enter numbers separated by space, or 0 for none)
```

1. Maple | Javor
2. Yew/Juniper | Tisa/Cempresi
3. Goosefoot/Pigweed | Pepeljuge/Stirovi
4. Plane Tree | Platan
5. Pine/Fir | Borovi/Jele
6. Mulberry | Dud
7. Hemp | Konoplja
8. Walnut | Orah
9. Beech | Bukva
10. Plantain | Bokvica
11. Mugwort | Pelen
12. Hornbeam | Grab
13. Nettle Tree | Koprovic
14. Alder | Jova
15. Birch | Breza
16. Poplar/Aspen | Topola
17. Willow | Vrba
18. Grass | Trave
19. Sorrel | Kiselica
20. Lime | Lipa
21. Oak | Hrast
22. Elm | Brest
23. Hazelnut | Leska
24. Nettle | Kopriva
25. Ragweed | Ambrozija
26. Ash | Jasen

0. No allergies

```
> 1 2 3 4 21 22 23 24 25 26
```

```
Welcome, natalija_br! Your account has been created.
```

4.3 Glavni meni

Nakon uspešne prijave ili registracije, korisniku se prikazuje glavni meni sa 9 opcija. U zaglavlju se prikazuju grad i korisničko ime prijavljenog korisnika.

```
=====
POLLENCAST | Beograd - Novi Beograd | natalija_br
=====
1. Today's pollen levels
2. 7-day forecast
3. Should I go outside today?
4. Top 5 cities lowest pollen
5. My allergy calendar
6. Allergen information
7. Edit profile
8. Logout
0. Exit
```

4.4 Trenutni nivo polena

Odabirom opcije 1, korisniku se prikazuju sve vrste polena sa vrednošću iznad 5 pz/m³ za korisnikov grad, sortirane od najveće ka najmanjoj vrednosti. Za svaku vrstu prikazuju se nazivi na srpskom i engleskom jeziku, današnja vrednost i nivo koncentracije (VERY HIGH, HIGH, MEDIUM, LOW ili NONE). Zasebno su prikazane i vrednosti alergena koje je korisnik izabrao pri registraciji.

=====			
TODAY'S POLLEN – Beograd - Novi Beograd			
=====			
Last measurement: 2026-05-04			
Species (EN)	Species (SR)	Value	Level

Mulberry	Dud	128	VERY HIGH
Pine/Fir	Borovi/Jele	44	MEDIUM
Nettle	Kopriva	31	MEDIUM
Birch	Breza	28	MEDIUM
Walnut	Orah	23	MEDIUM
Oak	Hrast	16	MEDIUM
Grass	Trave	13	MEDIUM
Plane Tree	Platan	9	LOW
Maple	Javor	7	LOW
Ash	Jasen	7	LOW
Beech	Bukva	2	LOW
Yew/Juniper	Tisa/Cempresi	1	LOW
Willow	Vrba	1	LOW
Your allergens:			

Nettle	Kopriva	31	MEDIUM
Oak	Hrast	16	MEDIUM
Plane Tree	Platan	9	LOW
Maple	Javor	7	LOW
Ash	Jasen	7	LOW
Yew/Juniper	Tisa/Cempresi	1	LOW
Goosefoot/Pigweed	Pepeljuge/Stirovi	0	NONE
Elm	Brest	0	NONE
Hazelnut	Leska	0	NONE
Ragweed	Ambrozija	0	NONE

4.5 7-dnevna prognoza polena

Odabirom opcije 2 iz glavnog menija, korisniku se prikazuje prognoza nivoa polena za njegov odabrani grad za narednih 7 dana. Za svaki dan prikazane su vrste polena sa vrednošću iznad 5 pz/m³, sortirane od najveće ka najmanjoj, kao i vrednosti svih korisnikovih alergena sa njihovim nivoima koncentracije.

2026-05-06			

Grass	Trave	32.3	MEDIUM
Pine/Fir	Borovi/Jele	24.2	MEDIUM
Oak	Hrast	12.9	MEDIUM
Mulberry	Dud	10.1	MEDIUM
Nettle	Kopriva	10.1	MEDIUM
--- Your allergens ---			
Plane Tree	Platan	2.0	LOW
Maple	Javor	0.5	LOW
Goosefoot/Pigweed	Pepeljuge/Stirovi	0.3	LOW
Yew/Juniper	Tisa/Cempresi	0.0	NONE
Mugwort	Pelen	0.0	NONE
Nettle Tree	Koprovic	0.0	NONE
Alder	Jova	0.0	NONE

4.6 Da li da izađem danas

Odabirom opcije 3 iz glavnog menija, korisniku se prikazuje personalizovana preporuka za današnji dan. Prikazuje se opšti nivo rizika označen bojom i tekstualni savet, a ispod se nalazi detaljan pregled svih korisnikovih alergena sa predviđenom koncentracijom polena, stepenom alergenosti i individualnim rizikom za svaku biljku.

=====				
SHOULD I GO OUTSIDE? – Kikinda				
=====				
Forecast for: 2026-05-06				
Overall Risk: ● Low				
Advice: Generally safe to go outside.				
Monitor how you feel.				
Your allergens breakdown:				

Species (EN)	Species (SR)	Value	Potency	Risk

Maple	Javor	0.5	low	● Low
Goosefoot/Pigweed	Pepeljuge/Stirovi	0.3	low to moderate	● Low
Plane Tree	Platan	2.0	moderate to high	● Low
Yew/Juniper	Tisa/Cempresi	0.0	moderate	✓ None
Mugwort	Pelen	0.0	high	✓ None
Nettle Tree	Koprovic	0.0	low	✓ None
Alder	Jova	0.0	very high	✓ None
Tip: Peak pollen hours are 5am-10am.				
Consider going out in the afternoon or after rain.				

4.7 Top 5 gradova sa najnižim polenom

Odabirom opcije 4 iz glavnog menija, prikazuje se rang lista 5 gradova sa najnižim ukupnim nivoom polena na osnovu predikcije neuronske mreže za današnji dan. Za svaki grad prikazuje se ukupna predviđena vrednost polena, kao i vrednosti korisnikovih alergena koji su prisutni u tom gradu.

```
=====
TOP 5 CITIES – LOWEST POLLEN
=====
Fetching data for all cities...

1. Vrsac – Total pollen: 0

2. Krusevac – Total pollen: 54
   Maple                4   LOW

3. Vranje – Total pollen: 136
   Maple                7   LOW
   Yew/Juniper          1   LOW
   Plane Tree           1   LOW

4. Vrbas – Total pollen: 226
   Plane Tree           6   LOW
   Yew/Juniper          2   LOW
   Goosefoot/Pigweed    1   LOW
   Alder                1   LOW

5. Beograd - Novi Beograd – Total pollen: 310
   Plane Tree           9   LOW
   Maple                7   LOW
   Yew/Juniper          1   LOW
```

4.8 Lični kalendar alergija

Odabirom opcije 5 iz glavnog menija, korisniku se prikazuje personalizovani kalendar alergija. U gornjem delu prikazuju se alergeni aktivni u trenutnom mesecu, sa nazivima na srpskom i engleskom jeziku, periodom cvetanja i stepenom alergenosti označenim bojom. Ispod se nalazi godišnji pregled po mesecima, gde je za svaki mesec vidljivo koji od korisnikovih alergena su aktivni i koliko su alergeni.

```
=====
MY ALLERGY CALENDAR
=====

Active allergens this month (May):
-----
Plane Tree      | Platan      | Late March - Mid May | ● Moderate to High
Goosefoot/Pigweed | Pepeljuge/Stirovi | May - October       | ● Low to Moderate
Maple           | Javor       | March - Late May     | ● Low
Nettle Tree     | Koprovic    | March - Late May     | ● Low

Full year overview:
-----

January
  No active allergens

February
  Alder           | Jova        | ● Very High
  Yew/Juniper     | Tisa/Cempresi | ● Moderate
```

4.9 Informacije o alergenu

Odabirom opcije 6 iz glavnog menija, korisniku se prikazuje lista svih 26 alergena sortirana po stepenu alergenosti, sa nazivima na engleskom i srpskom jeziku i stepenom alergenosti označenim bojom.

```
=====
ALLERGEN INFORMATION
=====

Select an allergen to learn more:

#   English      | Serbian      | Potency
-----
1.  Alder        | Jova         | ● Very High
2.  Birch        | Breza        | ● Very High
3.  Grass        | Trave        | ● Very High
4.  Ragweed      | Ambrozija    | ● Very High
5.  Mugwort      | Pelen        | ● High
6.  Plane Tree   | Platan       | ● Moderate to High
7.  Sorrel       | Kiselica     | ● Moderate to High
8.  Oak          | Hrast        | ● Moderate to High
```

Korisnik bira alergen unosom odgovarajućeg broja, nakon čega se prikazuju detalji o odabranoj biljci: tip biljke, sezona cvetanja, stepen alergenosti, opis, tipični simptomi i praktični saveti za smanjivanje izloženosti.

```
=====
Mugwort | Pelen
=====

Type:      Weed
Season:    Late July - October
Potency:   ● High

About:
  Mugwort is a common weed that pollinates in late summer. It cross-reacts with ragweed and certain foods.

Symptoms:
  Hay fever, asthma, oral allergy syndrome with celery, carrots, fennel and spices

Tips:
  Avoid celery, carrots, fennel and spices during mugwort season. Mugwort cross-reacts with ragweed pollen.
```

4.10 Izmena profila

Odabirom opcije 7 iz glavnog menija, korisniku se prikazuje meni za izmenu profila sa dve opcije: promena grada i promena profila alergija. Nakon izvršene izmene, promene se odmah primenjuju i korisnik nastavlja rad sa ažuriranim profilom.

```
--- EDIT PROFILE ---
1. Change city
2. Change allergy profile
0. Back
```

5. Implementacija

1. **core.clj** - predstavlja ulaznu tačku aplikacije. Sadrži **-main** funkciju koja se poziva pri pokretanju komandom **lein run**. Funkcija prikazuje zaglavlje aplikacije i pokreće petlju autentifikacije, gde se nakon odjave korisnika, aplikacija vraća na početni meni umesto da se zatvori.

```
(defn -main []  
  (cli/print-header)  
  (loop []  
    (let [user (cli/auth-menu)]  
      (when user  
        (cli/main-menu user)  
        (recur)))))
```

2. **cli.clj** - implementira konzolni interfejs aplikacije i predstavlja centralni modul koji povezuje sve ostale module. Sadrži sledeće funkcije:

- **read-int** - validira korisnički unos i automatski traži ponovni unos ako korisnik unese nešto što nije broj ili nije u dozvoljenom opsegu
- **select-city** - prikazuje 12 podržanih mesta i vraća naziv na ćirilici koji API razume
- **select-allergy-profile** - parsira unos poput "1 5 12" i pretvara ga u listu alergena korisnika
- **register** - proverava da li korisničko ime već postoji u bazi, a u slučaju uspešne registracije automatski prijavljuje korisnika i vraća ga na glavni meni
- **login** - pretražuje u bazi korisničko ime i vraća korisnika ili grešku ako ne postoji
- **auth-menu** - prikazuje početni meni sa opcijama za prijavu, registraciju i izlaz, u petlji sve dok korisnik ne uspe da se prijavi
- **show-today-pollen** - poziva forecast/get-forecast koji interno pokreće neuronsku mrežu, a zatim računanjem razlike između današnjeg datuma i datuma poslednjeg merenja određuje koji dan predikcije da prikaže
- **edit-profile** - omogućava promenu grada ili profila alergija, pri čemu se promene odmah primenjuju bez ponovne prijave
- **main-menu** - prikazuje glavni meni sa 9 opcija i upravlja tokom aplikacije

Na slici je prikazana funkcija **main-menu**. Funkcija koristi loop petlju sa korisničkim objektom kao parametrom, što omogućava da se promene profila odmah odraze u zaglavlju menija. Opcija za odjavu vraća nil što signalizira core.clj da vrati korisnika na početni meni za prijavu, dok opcija Exit zatvara aplikaciju pozivom System/exit.

```

(defn main-menu [user]
  (loop [user user]
    (println "\n=====")
    (println (str "  POLLENCAST | "
                  (get model/city-api->display (:city user) (:city user))
                  " | " (:username user))))
    (println "=====")
    (println "1. Today's pollen levels")
    (println "2. 7-day forecast")
    (println "3. Should I go outside today?")
    (println "4. Top 5 cities lowest pollen")
    (println "5. My allergy calendar")
    (println "6. Allergen information")
    (println "7. Edit profile")
    (println "8. Logout")
    (println "0. Exit")
    (case (read-int "\n> " #{0 1 2 3 4 5 6 7 8})
      1 (do (show-today-pollen user) (recur user))
      2 (do (forecast/show-forecast (:city user) (:allergy-profile user)) (recur user))
      3 (do (advice/show-advice user) (recur user))
      4 (do (cities/show-top5-lowest user) (recur user))
      5 (do (calendar/show-calendar user) (recur user))
      6 (do (allergen-info/show-allergen-info user) (recur user))
      7 (recur (edit-profile user))
      8 (do (println "\nLogged out successfully!") nil)
      0 (do (println "Goodbye!") (System/exit 0))))))

```

3. **data.clj** - obezbeđuje čuvanje korisničkih podataka kroz **Codax** biblioteku. Codax je key-value baza podataka pisana u Clojure-u koja sve podatke trajno upisuje na disk u folder **data/pollencast-db**. Sadrži sledeće funkcije:

- **database** - otvara konekciju ka lokalnoj bazi pri pokretanju aplikacije
- **create-user!** - kreira novog korisnika sa korisničkim imenom, gradom i profilom alergija
- **get-user** - pronalazi korisnika po korisničkom imenu i vraća sve njegove podatke
- **update-user!** - ažurira postojećeg korisnika, koristi se pri izmeni grada ili profila alergija

```

;; Open local database
(def database (db/open-database! "data/pollencast-db"))

;; ---- USERS ----

(defn create-user! [username city allergy-profile]
  (db/assoc-at! database [:users username]
    {:username      username
     :city          city
     :allergy-profile allergy-profile}))

(defn get-user [username]
  (db/get-at! database [:users username]))

(defn update-user! [username updates]
  (db/merge-at! database [:users username] updates))

```


4. **api.clj** - obezbeđuje komunikaciju sa API-jem Agencije za zaštitu životne sredine Srbije koji se nalazi na adresi **http://77.46.150.200**. Koristi **clj-http** biblioteku za slanje HTTP GET zahteva, pri čemu se JSON odgovori automatski pretvaraju u Clojure mape. Sadrži sledeće funkcije:

- **get-locations** - vraća listu svih gradova gde se nalaze stanice za merenje polena
- **get-allergens** - vraća listu svih alergena sa njihovim detaljima
- **get-pollen-by-year** - vraća sva merenja polena za određenu godinu za sve lokacije
- **get-pollen-for-location** - filtrira merenja za konkretan grad iz godišnjih podataka
- **get-last-30-days** - vraća poslednjih 30 dostupnih merenja za grad, koristi se kao ulaz za neuronsku mrežu
- **get-latest-pollen** - vraća poslednje dostupno merenje, koristi se za prikaz datuma poslednjeg merenja korisniku

Na slici su prikazani primeri funkcija **get-pollen-by-year** i **get-pollen-for-location**.

```
(defn get-pollen-by-year [year]
  (let [response (client/get (str base-url "/api/opendata/pollens/" year "/")
                             {:as :json})]
    (:body response)))

(defn get-pollen-for-location [year location-name]
  (let [all-data (get-pollen-by-year year)]
    (filter #(= (:location %) location-name) all-data)))
```

5. **allergens.clj** - sadrži podatke o svih 26 vrsta alergenog polena koje prati Agencija za zaštitu životne sredine Srbije. Podaci su organizovani kao Clojure mapa gde je svaka biljna vrsta predstavljena latiničnim nazivom (npr. :BETULA, :AMBROSIA) koji odgovara nazivu kolone u API odgovoru sa adrese **http://77.46.150.200**, što omogućava direktno mapiranje između API podataka i informacija o alergenu.

Za svaku vrstu su definisani naziv na srpskom i engleskom jeziku, tip biljke (drvo, trava ili korov), period cvetanja i stepen alergenosti. Svi podaci su preuzeti iz Excel tabele "Tipovi alergena" dostupne na platformi **dat.gov.rs**. Fajl sadrži i pomoćne funkcije **potency-label** i **type-label** koje pretvaraju keyword vrednosti u čitljive stringove za prikaz korisniku.

Na slici je prikazan primer nekoliko unosa iz **allergen-info** mape koji ilustruju strukturu podataka za svaku biljnu vrstu.

```
(def allergen-info
  {:AMBROSIA    {:name-sr "Ambrozija"
                  :name-en "Ragweed"
                  :type    :weed
                  :season  "July - October"
                  :potency :very-high}
   :PLANTAGO    {:name-sr "Bokvica"
                  :name-en "Plantain"
                  :type    :weed
                  :season  "May - September"
                  :potency :low-to-moderate}})
```

6. **allergen_info.clj** - proširuje podatke iz **allergens.clj** detaljnim opisima za svaku od 26 biljnih vrsta. Za svaki alergen definisani su: kratak naučni opis biljke i njenih karakteristika u kontekstu Srbije, tipični alergijski simptomi koje izaziva i praktični saveti za smanjivanje izloženosti. Podaci su prikupljeni sa Wikipedia i European Aeroallergen Network platforme.

Funkcija **show-allergen-info** prikazuje listu svih alergena sortiranu po stepenu alergenosti od najvišeg ka najnižem, a nakon što korisnik odabere alergen, prikazuju se svi detalji o odabranoj biljci.

7. **preprocess.clj** - priprema podatke za treniranje i korišćenje neuronske mreže. Sadrži sledeće funkcije:
- **record->raw** - pretvara svaki API zapis u vektor od 32 vrednosti: 26 vrednosti koncentracija polena i 6 vremenskih karakteristika
 - **cyclic-encode** - predstavlja datum kroz sinusnu i kosinusnu funkciju meseca, dana u godini i dana u sedmici
 - **compute-stats** - izračunava srednju vrednost i standardnu devijaciju iz trening podataka
 - **normalize** i **normalize-record** - primenjuju Z-score normalizaciju na vrednosti polena
 - **denormalize** - pretvara normalizovane vrednosti nazad u originalne jedinice (pz/m3)

- **active-season** - filtrira zapise van perioda mart-oktobar kako bi se izbegao problem nultih vrednosti tokom zimskih meseci
- **load-training-data** - učitava podatke sa API-ja za period od 2019. do 2025. godine
- **create-windows** - od hronološki uređenih zapisa formira parove ulaz-izlaz metodom klznog prozora, gde ulaz čini 30 uzastopnih dana, a izlaz 21 dana
- **prepare-training-data** - koordinira ceo proces pripreme podataka: učitava podatke, filtrira aktivnu sezonu, normalizuje vrednosti i kreira prozore za treniranje
- **next-dates** - generiše listu narednih n datuma, koristi se pri prikazu prognoza

Na slici je prikazana funkcija **prepare-training-data** koja prikazuje proces pripreme podataka.

```
(defn prepare-training-data
  ([location] (prepare-training-data location 30 21))
  ([location window-size pred-size]
   (let [data (load-training-data location)
         active (vec (filter active-season? data))
         s (compute-stats active)
         _ (reset! stats s)
         _ (println (str "Stats - mean: " (format "%.4f" (double (:mean s)))
                        " std: " (format "%.4f" (double (:std s)))))
         normed (mapv #(normalize-record % (:mean s) (:std s)) active)
         windows (create-windows normed window-size pred-size)]
     (println (str "Active season records: " (count active)))
     (println (str "Prepared " (count windows)
                  " training windows for " location))
     windows)))
```

8. **model.clj** - definiše arhitekturu neuronske mreže i sadrži pomoćne konstante i mape za rad sa gradovima. Sadrži sledeće definicije:

- **input-size, output-size, batch-size** - konstante koje definišu dimenzije mreže: ulaz je 960 vrednosti (30 dana $\times$ 32 karakteristike), izlaz je 546 vrednosti (21 dan $\times$ 26 vrsta polena), a batch size je 32

- **create-network** - definiše arhitekturu mreže sa tri skrivena sloja od 512, 256 i 128 neurona sa ReLU aktivacijom i dropout regularizacijom od 30%, i linearnim izlaznim slojem
- **flatten-input** i **flatten-output** - pretvaraju listu vektora u jedan ravan vektor pogodan za ulaz u mrežu
- **supported-cities** - lista podržanih gradova na ćirilici koja se koristi pri pozivu API-ja
- **city-display->api** - mapa koja prevodi latinične nazive gradova u ćirilčne za API pozive
- **city-api->display** - inverzna mapa koja prevodi ćirilčne nazive u latinične za prikaz korisniku, dobijena automatski invertovanjem city-display->api

Na slici je prikazana funkcija **create-network** koja definiše arhitekturu neuronske mreže.

```
(defn create-network
  ([] (create-network input-size output-size))
  ([in-size out-size]
   (let [net-spec [(fully-connected [512] :relu)
                    (dropout 0.3)
                    (fully-connected [256] :relu)
                    (dropout 0.3)
                    (fully-connected [128] :relu)
                    (fully-connected [out-size] :linear)]]
     (network (desc [batch-size in-size] :float :nc)
               net-spec))))
```

9. **train.clj** - implementira proces treniranja neuronske mreže za svaki podržani grad.

Sadrži sledeće funkcije:

- **windows->matrices** - pretvara listu kliznih prozora iz **preprocess.clj** u dve float matrice: ulaznu matricu X dimenzija (broj_prozora $\times$ 960) i izlaznu matricu Y dimenzija (broj_prozora $\times$ 546)
- **train-model!** - koordinira proces treniranja: poziva **prepare-training-data** da pripremi podatke, deli ih na 80% za treniranje i 20% za validaciju, kreira i inicijalizuje mrežu Adam optimizatorom, prenosi podatke u tenzore i pokreće petlju treniranja.

Trening se odvija u rundama od po 10 epoha, a nakon svake runde ispisuje se vrednost greške i RMSE u pz/m3. Nakon završetka treniranja, model se čuva na disk pozivom **persist/save-model!** u folder **models/** kao binarna datoteka (npr. **НННН.bin**) zajedno sa statistikama normalizacije (npr. **НННН_stats.edn**).

Na slici je prikazana funkcija **train-model!** koja pokazuje tok procesa treniranja od učitavanja podataka do čuvanja modela.

```
(defn train-model! [location epochs]
  (println (str "Loading training data for " location "..."))
  (let [windows (prep/prepare-training-data location)
        n-train (int (* 0.8 (count windows)))
        train-w (vec (take n-train windows))]
    (println (str "Training samples: " n-train))
    (println "Preparing matrices...")
    (let [{:keys [x y n]} (windows->matrices train-w)
          net-bp (model/create-network)
          net (init! (net-bp :adam))
          x-tz (tensor [n model/input-size] :float :nc)
          y-tz (tensor [n model/output-size] :float :nc)]
      (transfer! x x-tz)
      (transfer! y y-tz)
      (println "Starting training...")
      (doseq [round (range (quot epochs 10))]
        (let [loss (train! net x-tz y-tz :quadratic 10 [])
              std (double (:std @prep/stats))
              rmse (* (Math/sqrt (double loss)) std)]
          (println (str "Epochs " (* round 10) "-" (* (inc round) 10)
                        " | loss: " (format "%.6f" (double loss))
                        " | RMSE: " (format "%.2f" rmse)
                        " pollen/m³"))))
      (println "Training complete!")
      (persist/save-model! net location)
      net)))
```

10. **persistence.clj** - čuva i učitava istrenirane modele sa diska. Sadrži sledeće funkcije:

- **model-path** i **stats-path** - generišu putanje do fajlova modela i statistika na osnovu naziva grada, npr. **models/НННН.bin** i **models/НННН_stats.edn**
- **model-exists?** - proverava da li model za dati grad već postoji na disku

- **open-channel-rw** i **open-channel-new** - otvaraju Java FileChannel za čitanje i pisanje
- **save-model!** - čuva model na disk u dva koraka: statistike normalizacije (mean i std) se čuvaju kao EDN tekstualni fajl, a težine mreže se serijalizuju kroz FileChannel pozivom **transfer!**
- **load-model!** - učitava model sa diska u obrnutom redosledu: najpre učitava statistike i postavlja ih u globalni stats atom u **preprocess.clj**, a zatim kreira novu mrežu, otvara kanal i učitava težine

Na slici su prikazane funkcije **save-model!** i **load-model!** koje ilustruju kako se model čuva i učitava sa diska.

```
(defn save-model!
  ([net city] (save-model! net city ""))
  ([net city suffix]
   (io/make-parents (model-path city suffix))
   (spit (stats-path city suffix) (pr-str @prep/stats))
   (with-open [ch (open-channel-new (model-path city suffix))]
    (transfer! net ch))
   (println (str "Model saved: " (model-path city suffix)))))

(defn load-model!
  ([city] (load-model! city ""))
  ([city suffix]
   (when (model-exists? city suffix)
    (println (str "Loading model for " city "..."))
    (let [stats (read-string (slurp (stats-path city suffix)))]
      (reset! prep/stats stats))
    (let [net-bp (model/create-network)
          net    (init! (net-bp :adam))]
      (with-open [ch (open-channel-rw (model-path city suffix))]
        (transfer! ch net))
      (println (str "Model loaded for " city))
      net)))))
```

11. **forecast.clj** - generiše prognozu polena korišćenjem istreniranog modela. Sadrži sledeće funkcije:

- **loaded-models** - atom koji čuva učitane modele u memoriji kako bi se izbeglo ponavljano učitavanje sa diska pri svakom pozivu prognoze

- **get-model** - proverava da li je model već u memoriji, a ako nije, učitava ga sa diska i čuva u loaded-models
- **pollen-level** - pretvara numeričku vrednost koncentracije u tekstualni nivo (NONE, LOW, MEDIUM, HIGH, VERY HIGH)
- **prepare-input** - učitava poslednjih 30 dana merenja sa API-ja, normalizuje ih i pretvara u Deep Diamond tenzor pogodan za ulaz u mrežu
- **parse-predictions** - pretvara sirovi izlaz mreže (vektor od 546 vrednosti) u listu od 21 dana, gde svaki dan sadrži datum i vrednosti za 26 vrsta polena; negativne vrednosti se postavljaju na nulu jer koncentracija polena ne može biti negativna
- **get-forecast** - učitava model, priprema ulaz, poziva infer! iz Deep Diamond biblioteke za izvođenje predikcije i parsira rezultat
- **show-forecast** - prikazuje 7 dana prognoze počevši od današnjeg datuma, uzimajući u obzir kašnjenje API-ja računanjem razlike između današnjeg datuma i datuma poslednjeg merenja

Na slici je prikazana funkcija **get-forecast** koja pokazuje kako se model učitava, ulaz priprema i predikcija izvodi.

```
(defn get-forecast [city]
  (let [net (get-model city)]
    (if (nil? net)
      (do (println (str "No model for " city ". Train first. "))
          nil)
      (when-let [input (prepare-input city)]
        (let [out-tz (infer! net (:tensor input))
              raw    (float-array (* model/batch-size model/output-size))
              _       (transfer! out-tz raw)
              result  (vec raw)
              forecast (parse-predictions result (:last-date input) @prep/stats)]
          forecast))))))
```

12. **advice.clj** - implementira algoritam za računanje personalnog rizika od alergija za današnji dan. Sadrži sledeće funkcije:

- **potency-weight** - dodeljuje numeričku težinu svakom stepenu alergenosti, od 0.5 za Very Low do 5.0 za Very High, na osnovu podataka iz resursa "Tipovi alergena" sa platforme dat.gov.rs

- **pollen-category** - svrstava koncentraciju polena u kategoriju: vrednosti ispod 30 pz/m3 za Very High biljke (npr. ambrozija) ili ispod 60 pz/m3 za ostale biljke se svrstavaju u LOW, između praga i 100 pz/m3 u MEDIUM, a iznad 100 pz/m3 u HIGH
- **individual-risk** - kombinuje kategoriju polena i težinu alergenosti da bi izračunala individualni rizik za svaki alergen
- **risk-rank** - dodeljuje numeričku vrednost nivou rizika, koristi se za sortiranje
- **risk-emoji** i **overall-advice** - formatiraju nivo rizika i odgovarajući savet za prikaz korisniku
- **calculate-risk** - za svaki korisnikov alergen poziva **individual-risk** i određuje ukupni rizik kao najgori pojedinačni rizik
- **show-advice** - poziva **forecast/get-forecast**, određuje koji dan predikcije odgovara današnjem datumu, poziva **calculate-risk** i prikazuje rezultat korisniku

Na slici je prikazana funkcija **calculate-risk** koja određuje ukupni rizik.

```
(defn calculate-risk [entry allergy-profile]
  (if (empty? allergy-profile)
      {:overall :none :details []}
      (let [details (for [allergen allergy-profile]
                       (let [allergen-kw (if (keyword? allergen)
                                              allergen
                                              (keyword (name allergen)))
                            value         (double (get entry allergen-kw 0))
                            info          (get allergens/allergen-info allergen-kw)
                            potency       (:potency info)
                            risk          (individual-risk value potency)]
                         {:allergen allergen-kw
                          :name-en  (:name-en info)
                          :name-sr  (:name-sr info)
                          :value     value
                          :potency   potency
                          :risk      risk}))
            sorted (sort-by #(risk-rank (:risk %)) > details)
            worst  (first sorted)]
          {:overall (:risk worst)
           :details sorted})))
```

13. **calendar.clj** - implementira prikaz ličnog alergijskog kalendara korisnika. Sadrži sledeće funkcije:

- **current-month** - vraća broj trenutnog meseca korišćenjem Java LocalDate klase
- **potency-rank** i **potency-circle** - dodeljuju numeričku vrednost i emoji oznaku svakom stepenu alergenosti, koriste se za sortiranje i prikaz

- **parse-season-months** - parsira tekstualni opis sezone cvetanja (npr. "Late March - Mid May") u početni i krajnji mesec
- **active-this-month?** - na osnovu parsiranog perioda cvetanja proverava da li je određeni alergen aktivan u datom mesecu
- **sort-by-potency** - sortira listu alergena od najvišeg ka najnižem stepenu alergenosti
- **show-calendar** - prikazuje dve celine: alergene aktivne u trenutnom mesecu sa periodom cvetanja i stepenom alergenosti, i godišnji pregled po mesecima gde je tekući mesec označen strelicom

14. **cities.clj** - implementira poređenje nivoa polena između podržanih gradova za današnji dan. Sadrži sledeće funkcije:

- **pollen-level** - pretvara numeričku vrednost koncentracije u tekstualni nivo (NONE, LOW, MEDIUM, HIGH, VERY HIGH)
- **city-today-pollen** - za svaki grad poziva **forecast/get-forecast**, određuje koji dan predikcije odgovara današnjem datumu istim mehanizmom kao u **forecast.clj** i **advice.clj**, i vraća ukupnu sumu svih 26 vrednosti polena za taj dan
- **show-top5-lowest** - poziva **city-today-pollen** za svih 12 podržanih gradova, sortira ih po ukupnoj vrednosti polena od najmanje ka najvećoj i prikazuje prvih 5. Za svaki grad dodatno prikazuje vrednosti korisnikovih alergena sa vrednošću iznad 0.1 pz/m³, sortirane od najveće ka manjoj

Na slici je prikazana funkcija **city-today-pollen** koja ilustruje kako se predikcija koristi za poređenje gradova.

```

(defn city-today-pollen [city]
  (let [fc (forecast/get-forecast city)]
    (if (nil? fc)
      {:city city :total 0 :entry nil}
      (let [formatter (java.time.format.DateTimeFormatter/ofPattern "yyyy-MM-dd")
            first-ld (java.time.LocalDate/parse (:date (first fc)) formatter)
            last-meas (.minusDays first-ld 1)
            today-ld (java.time.LocalDate/now)
            lag (.between java.time.temporal.ChronoUnit/DAYS last-meas today-ld)
            idx (int (min (max (dec lag) 0) 20))
            today-data (nth fc idx)
            values (:values today-data)
            entry (zipmap prep/pollen-species values)
            total (reduce + values)]
        {:city city
         :total total
         :entry entry}))))

```

6. Testiranje

Za testiranje aplikacije korišćen je Clojure-ov ugrađeni okvir za testiranje **clojure.test**. Testovi se nalaze u fajlu **test/pollen_cast/core_test.clj** i mogu se pokrenuti komandom **lein test**.

Implementirani su sledeći testovi:

- **test-normalize** i **test-denormalize** - proveravaju ispravnost Z-score normalizacije i denormalizacije vrednosti
- **test-cyclic-encode** - proverava da ciklično kodiranje vraća vrednosti između -1 i 1 i da su januar i decembar bliske vrednosti u kodiranom prostoru
- **test-date-features** - proverava da funkcija generiše tačno 6 datumskih karakteristika u ispravnom opsegu
- **test-active-season** - proverava da su mart i oktobar prepoznati kao aktivna sezona, a januar i decembar kao neaktivni meseci
- **test-pollen-category** - proverava ispravnost kategorizacije koncentracije polena
- **test-individual-risk** - proverava računanje individualnog rizika za različite kombinacije koncentracije i alergenosti
- **test-risk-rank** - proverava da su nivoi rizika ispravno rangirani
- **test-active-this-month** - proverava da funkcija ispravno određuje da li je alergen aktivan u datom mesecu
- **test-allergen-info-complete** - proverava da sve 26 vrste polena imaju definisane podatke u allergens.clj
- **test-pollen-species-count** - proverava da lista pollen-species sadrži tačno 26 vrsta